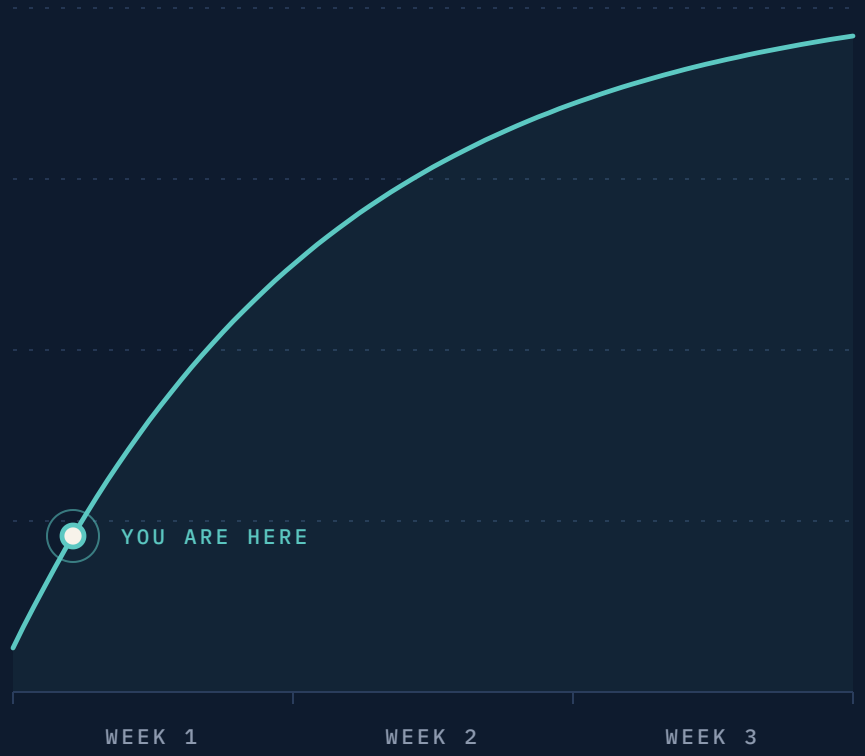
WEEK 1 · DAY 2 · OVERVIEW

Bagging and Out-of-Bag Scores



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 1 averaged ten full trees by hand and reached a validation ROC AUC of 0.7352, short of Course 4's 12-leaf pruned tree at 0.7627.

The desk now wants hundreds of trees, and a rule for setting them.

Course 4 chose its settings on the 6,000 validation rows; Course 5 chooses nothing on them.

So every choice needs a score made inside the 18,000 training rows that does not flatter the model, and a bag of trees carries one for free.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Fit a bagged model with scikit-learn and check it against an average you take yourself, and watch an account's output and the bag's score settle as members are added.

Score a bag on its own out-of-bag rows, read every number it reports, compare an out-of-bag score with five-fold cross-validation, choose a setting out of bag and report it on validation rows, and bag a model other than a tree.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 2

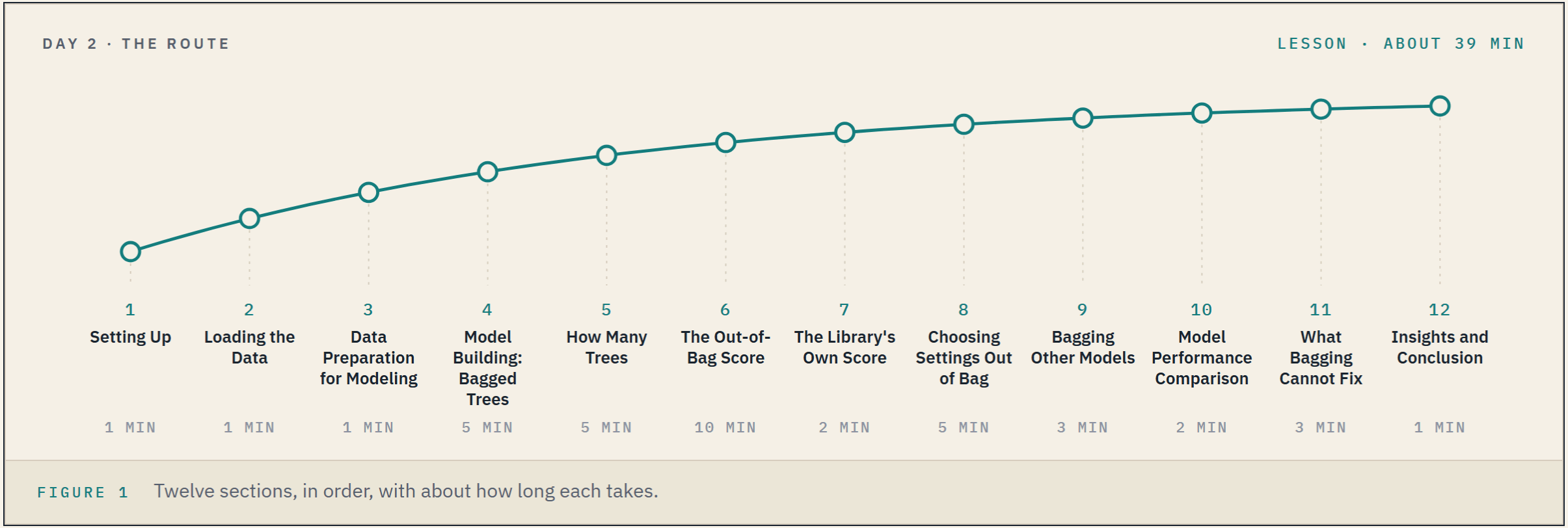


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 39 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import BaggingClassifier
```

Read the card file and count the defaults.

Loading the Data

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
print(len(FEATURES), "features")
```

```
19 features
```

THE SHAPE

```
print(card.shape)
```

```
(30000, 25)
```

Rebuild Course 4's training, validation and test rows.

Data Preparation for Modeling

COURSE 4'S SPLIT

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

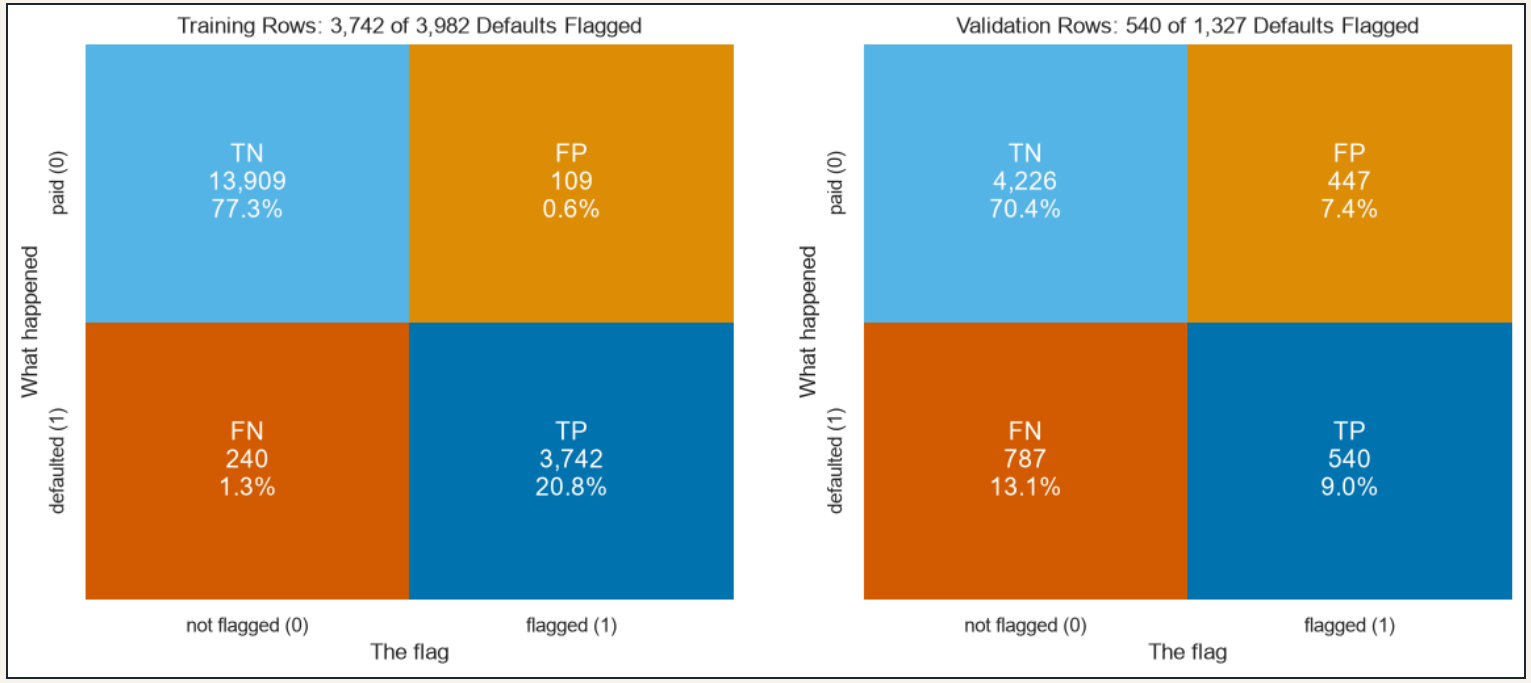
WHO DOES WHAT

The same 18,000, 6,000 and 6,000 accounts as Course 4, each with the same default share.

Course 4 used the validation rows to choose settings; Course 5 only reports on them, and never scores the test rows.

Training rows fit, validation rows report, test rows stay closed.

Model Building: Bagged Trees



TRAINING AGAINST VALIDATION

	TRAINING	VALIDATION
ROC AUC	0.9977	0.7320
Accuracy	0.9806	0.7943
Recall	0.9397	0.4069
Precision	0.9717	0.5471
F1	0.9554	0.4667

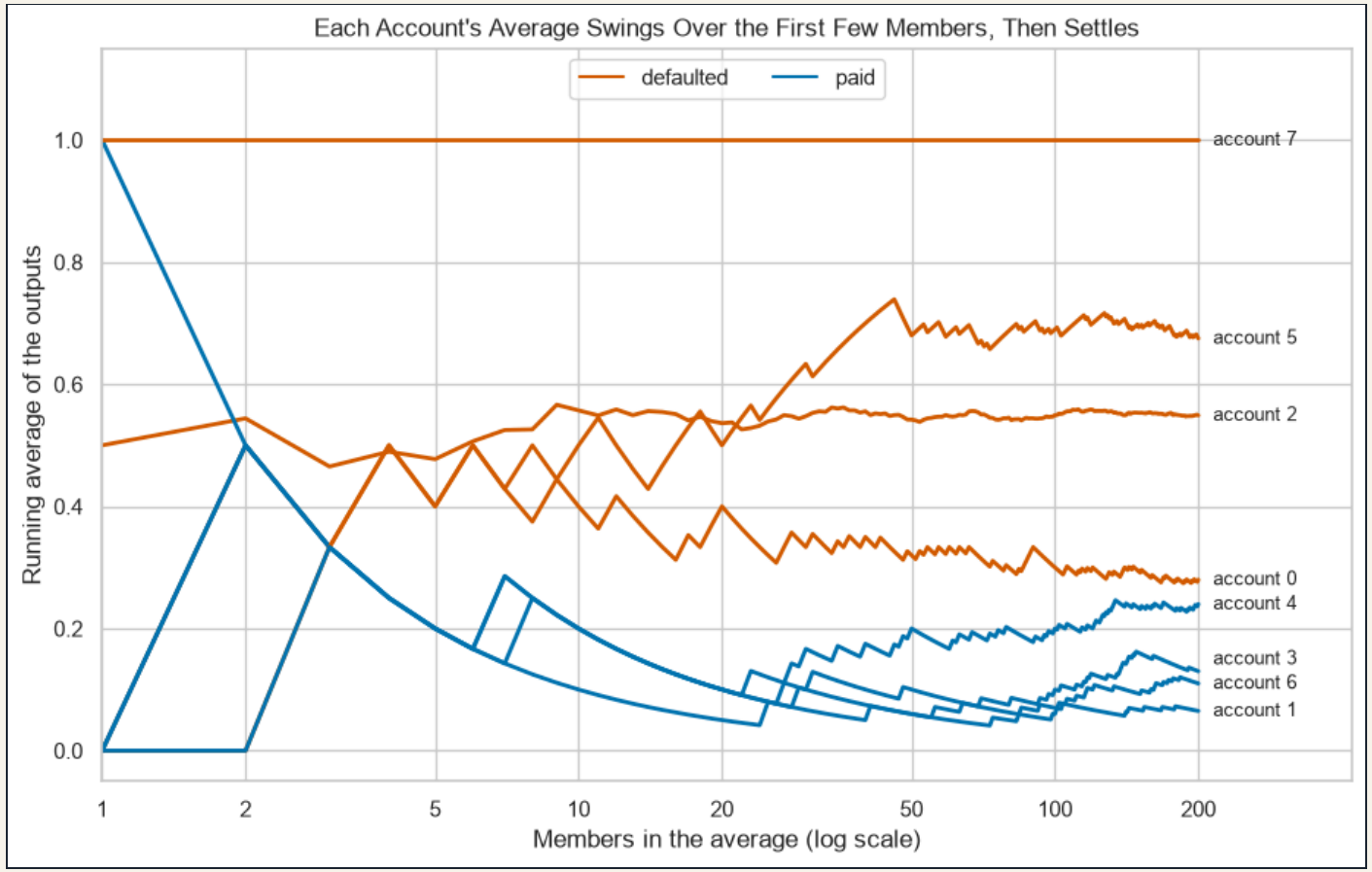
TEN TREES IN ONE CALL

`BaggingClassifier` does Day 1's work in one call: it draws **`n_estimators`** bootstrap samples, fits a copy of **`estimator`** on each, and averages their outputs.

A bagged probability is the plain average of its members, nothing more.

Grow 200 trees and track each running average.

How Many Trees



READING THE TRACES

Account 4 paid: its first member said 1.0, and its average settles near 0.24.

Account 7 defaulted, and every member says 1.0.

One more member moves an account less and less, in step with one over the count of members: 0.13 at the second, 0.026 at the 10th, 0.0013 at the 200th.

51 accounts still change flag between 100 and 200 members.

Every trace swings over the first few members, then flattens.

The Out-of-Bag Score

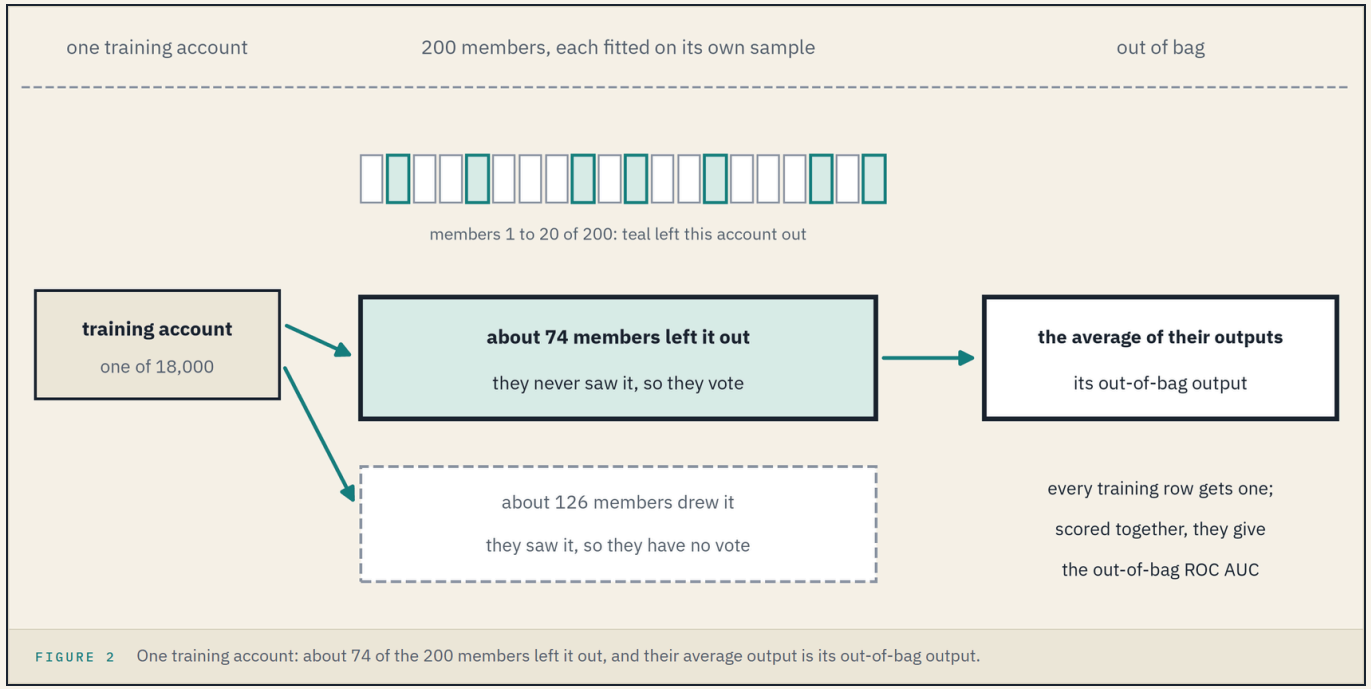


FIGURE 2 One training account: about 74 of the 200 members left it out, and their average output is its out-of-bag output.

MARKING WHO LEFT EACH ROW OUT

```
left_out = np.ones((len(X_train), 200), dtype=bool)
for j, rows in enumerate(bag.estimators_samples_):
    left_out[rows, j] = False
```

THE AVERAGE BY HAND

```
Largest gap from scikit-learn's: 3.3306690738754696e-16
```

A row's out-of-bag output is the average of only the members whose samples left it out.

The Library's Own Score

bag.oob_score_ is the one out-of-bag number scikit-learn keeps for a bag.

THE LINE THAT ASKS

```
print("oob_score_:", round(bag.oob_score_, 4))
```

WHAT THE BAG KEEPS

oob_score=True makes the bag keep a second set of outputs while it fits; we open them in a later section.

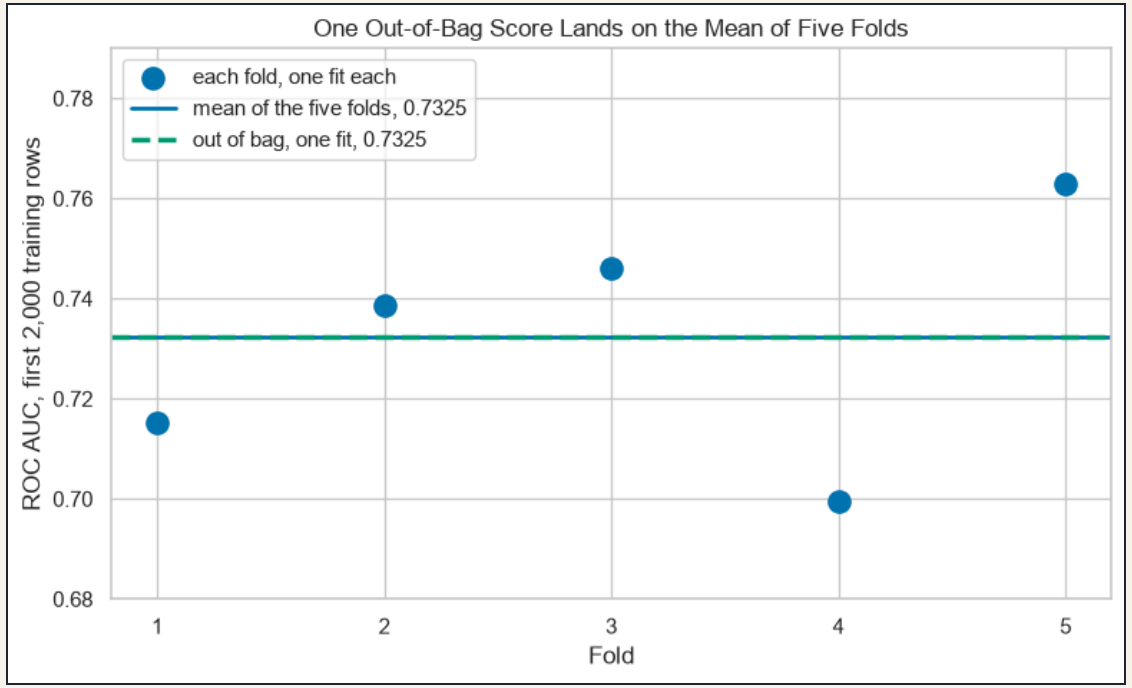
THE QUESTION

Q. What is it for **bag**? Write down a number.

THE FIT THAT KEEPS IT

```
bag = BaggingClassifier(estimator=DecisionTreeClassifier(random_state=SEED), n_estimators=200, oob_score=True,
                        random_state=SEED, n_jobs=1).fit(X_train, y_train)
```

Choosing Settings Out of Bag



FIVE FOLDS

```
Folds: [0.7153 0.7385 0.746  0.6994 0.763 ]  Mean: 0.7325  Sd: 0.0225
```

THE BAG'S SETTINGS

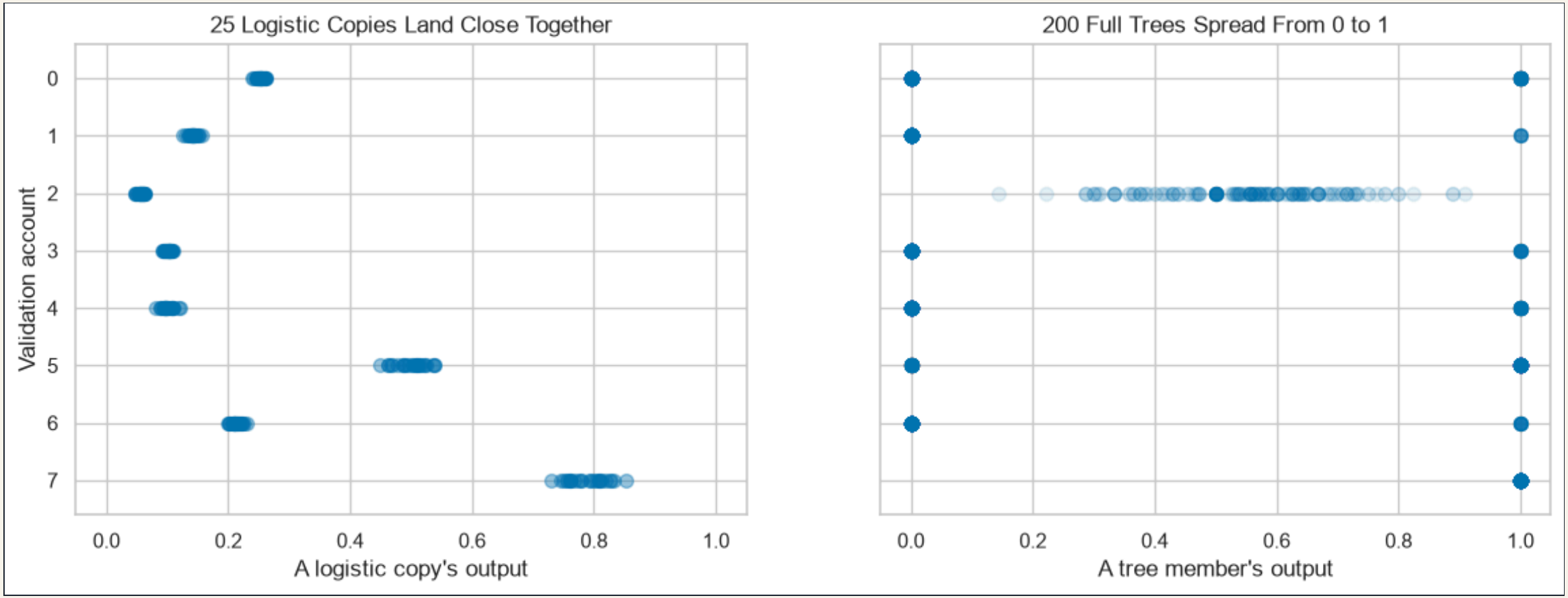
```
bag.get_params(deep=False)
```

```
{'bootstrap': True,
 'estimator': DecisionTreeClassifier(random_state=42),
 'max_samples': None,
 'n_estimators': 200,
 'oob_score': True,
```

On a bag, one out-of-bag score stands in for five folds, from one fit instead of five.

Bag a logistic regression and a weighted tree.

Bagging Other Models



Bagging helps a model that moves with its sample; the tree inside carries its own settings, weights included.

Model Performance Comparison

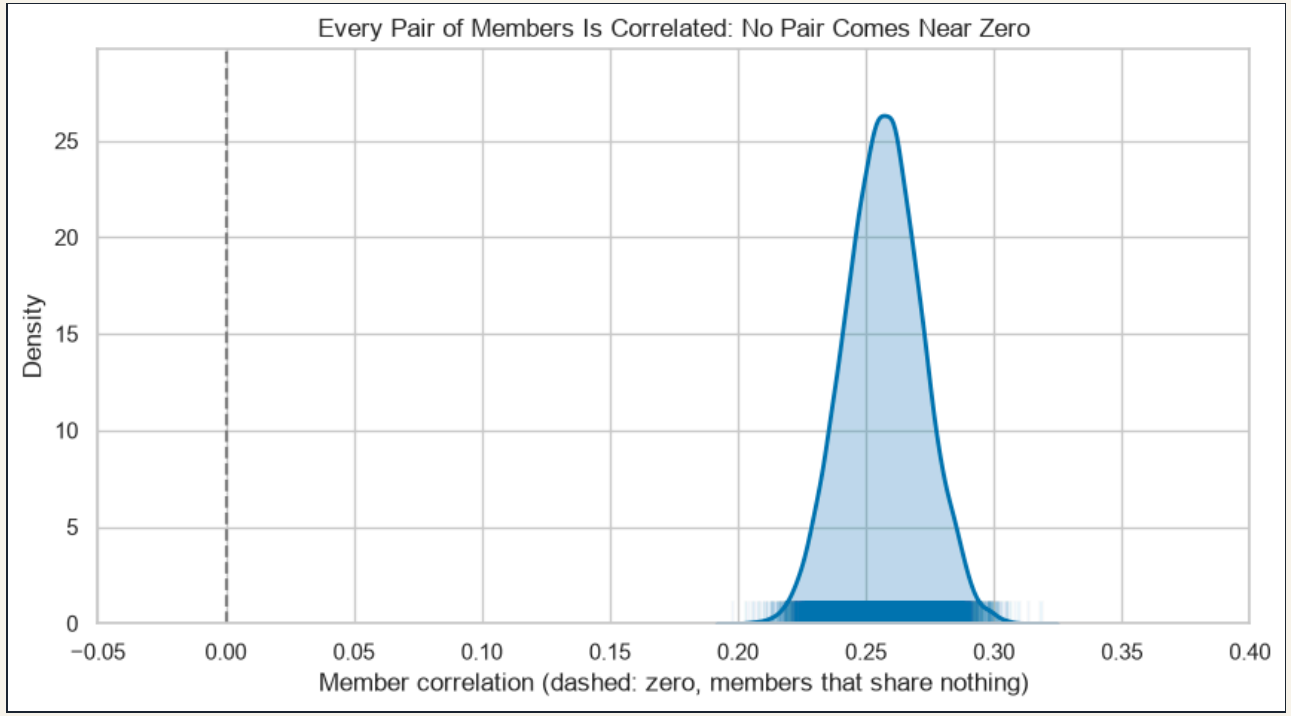
	ROC AUC	ACCURACY	RECALL	PRECISION	F1	AVERAGE PRECISION
Course 4 full tree	0.6161	0.7278	0.4032	0.3888	0.3959	0.2917
Course 4 pruned tree	0.7627	0.7683	0.5840	0.4805	0.5272	0.4954
Course 4 logistic	0.7190	0.8123	0.2442	0.7248	0.3653	0.4946
bag of 10	0.7320	0.7943	0.4069	0.5471	0.4667	0.4584
chosen bag	0.7837	0.8202	0.3399	0.6896	0.4553	0.5573
bagged logistic	0.7195	0.8117	0.2411	0.7223	0.3616	0.4952
weighted bag	0.7796	0.7692	0.6142	0.4828	0.5406	0.5543

READING THE TABLE

Accuracy, recall and precision depend on the 0.5 threshold: the weighted bag flags far more and so has the highest recall.

One bag, chosen out of bag, leads. Every bag still asks its trees the same thing first.

What Bagging Cannot Fix



THE FIRST QUESTION OF 200 MEMBERS

```
PAY_0 <= 1.5    199
PAY_0 <= 0.5      1
Name: count, dtype: int64
```

EVERY PAIR OF MEMBERS

```
19900 pairs  Mean: 0.2567  Lowest: 0.198  Highest: 0.318
```

To gain more, the members must disagree more without getting weaker. Day 3 trades some of each tree's strength for disagreement.

Insights and Conclusion



ROWS WITH NO VOTE YET

```
{1: 11413, 2: 7226, 5: 1822, 10: 186, 20: 1, 21: 0}
```

After 10 members, 186 rows still have none; from 21 members on, every row has a vote.

Day 3 makes the trees disagree on purpose.

You Can Now

- Fit a **`BaggingClassifier`** and check its output against the average of its members.
- Watch an account's running average and the bag's score as members are added.
- Find each training row's out-of-bag output and score it, by hand and with **`oob_decision_function_`**.
- Say what **`oob_score_`** measures, and compare an out-of-bag score with five folds.
- Choose a member's setting out of bag, report it on validation rows, and bag a model other than a tree.
- Measure how alike a bag's members are.

NEXT
WEEK 1 · DAY 3

Random Forests

Day 3 makes the trees disagree on purpose: the random forest.